1. Client-Side Rendering (CSR)

☑ Simple Explanation

In CSR, the browser loads a basic page first, and then JavaScript fetches data and updates the page dynamically.

⚙ Technical Explanation

CSR relies on JavaScript running in the browser. After the initial HTML is loaded, API calls are made using hooks like **useEffect**, and the DOM is updated dynamically. No pre-rendered HTML is sent from the server.

📌 Key Points

- Rendering happens in browser
- Uses JavaScript for data fetching
- Not SEO-friendly initially

📄 Conclusion

👉 CSR is best for **interactive applications**, but it has slower initial loading and weaker SEO.

2. Static Site Generation (SSG)

☑ Simple Explanation

Static Site Generation means that the web page is created **once at build time** and then served to every user as a ready-made HTML file. The content does not change unless the site is rebuilt.

⚙ Technical Explanation

In SSG, Next.js pre-renders pages during the build process using functions like **getStaticProps**. The generated HTML and JSON files are stored on a CDN and served to users without executing server-side code on each request. This improves performance and reduces server load.

📌 Key Points

- Pre-rendered at build time
- Same content for all users
- No server computation per request

📝 Conclusion

👉 SSG provides **high performance and SEO benefits**, but it is not suitable for frequently changing data.

```
export async function getStaticProps() {
 const res = await fetch("https://api.example.com/posts");
 const data = await res.json();
 return {
  props: { data },
 }; }
```

3. Server-Side Rendering (SSR)

☑ Simple Explanation

In SSR, the page is generated **every time a user makes a request**, so the user always gets the latest data.

⚙ Technical Explanation

SSR uses **getServerSideProps** to fetch data on each request. The server processes the request, fetches data, generates HTML dynamically, and sends it to the client. This ensures real-time data rendering.

📌 Key Points

- Rendered on every request
- Always fresh data
- Higher server usage

📋 Conclusion

👉 SSR is suitable for **real-time and personalized applications**, but it increases server load and response time.

export async function **getServerSideProps()** {

const res = await fetch("API");

const data = await res.json();

return {

```
  props: { data },
};
}
```

4. Incremental Static Regeneration (ISR)

☑ Simple Explanation

ISR is an advanced version of static generation where the page is updated **automatically after a certain time**, without rebuilding the entire site.

⚙ Technical Explanation

ISR allows Next.js to regenerate static pages in the background using the ***revalidate*** property. When the revalidation time expires, the next request triggers a background rebuild, and updated content is served without downtime.

📌 Key Points

- Static pages with automatic updates
- Background regeneration
- No full rebuild required

📝 Conclusion

👉 ISR combines **performance of SSG with updated data**, making it ideal for dynamic content with moderate changes.

```
export async function getStaticProps() {
  const res = await fetch("API");
```

```
 const data = await res.json();

 return {

  props: { data },

  revalidate: 10,  // 10 seconds

 }; }
```

5. Streaming and Suspense Rendering

☑ Simple Explanation

Streaming allows the page to load in **small parts instead of waiting for the entire page**, improving user experience.

⚙ Technical Explanation

Streaming uses React **Suspense** to divide the UI into components. The server streams HTML progressively, sending ready parts first and loading slower components later using fallback UI.

📌 Key Points

- Partial rendering
- Uses Suspense boundaries
- Improves perceived performance

📄 Conclusion

👉 Streaming enhances **user experience by reducing waiting time**, especially in large applications.

<Suspense fallback={ <p>Loading...</p> }>

<SlowComponent />

</Suspense>

6. Edge Rendering

☑ Simple Explanation

Edge Rendering means the page is generated **closer to the user's location**, making it faster.

⚙ Technical Explanation

Edge Rendering runs code on distributed CDN edge servers instead of a central server. It uses the Edge Runtime, which reduces latency and improves response time for global users.

📌 Key Points

- Runs on edge servers
- Low latency
- Limited Node.js support

📝 Conclusion

👉 Edge Rendering provides **fast global performance**, but has some runtime limitations.

export const **runtime** = "**edge**";

7. Hybrid Rendering

☑ Simple Explanation

Hybrid Rendering means using **multiple rendering techniques in one application**.

⚙ Technical Explanation

Next.js allows combining SSG, SSR, CSR, and other rendering methods within the same project. Each page or component can use a different strategy based on requirements.

📌 Key Points

- Combination of rendering methods
- Flexible architecture
- Optimized performance

📄 Conclusion

👉 Hybrid Rendering is the **most practical approach**, as it balances performance and flexibility.